# ECG Heartbeat Classification using a 1D CNN

Sohaib Khalaf
760012524

## 1 Introduction

For this coursework, I trained a deep learning model to classify ECG heartbeat samples into five classes: Normal, Hypersensitive, Type 1, Type 2, and Type 3. ECG signals record the electrical activity of the heart, so classifying heartbeat patterns can help detect abnormal cases such as arrhythmias or myocardial infarction [1]. This is a supervised multiclass classification task. The model receives ECG features and predicts one heartbeat label. I treated the task as $y = ANN(X)$, where $X$ is the ECG input and $y$ is the predicted class.

The training set contained 87,554 samples and the test set contained 21,892 samples. Each row had 187 ECG signal features and one label. The coursework brief refers to 188 columns, but the last column is the target label, so the model input used 187 features. Since ECG signals are one-dimensional waveform data, I chose a 1D convolutional neural network instead of a normal dense network. A 1D CNN can scan across the heartbeat sequence and pick up local signal patterns, which fits the way ECG data is stored [2, 3, 4].

The first issue I noticed was the class imbalance. The Normal class had 72,471 training samples, about 82.8% of the training set, while Type 2 had only 641 samples, about 0.7%. This meant a model could get a misleading accuracy by focusing on Normal heartbeats and ignoring rare classes. To deal with this, the final pipeline used balanced class weights, shuffling, hyperparameter tuning, and Stratified K-Fold cross-validation. The final matched notebook result was 87.83% test accuracy and a test macro ROC-AUC of 0.9890.

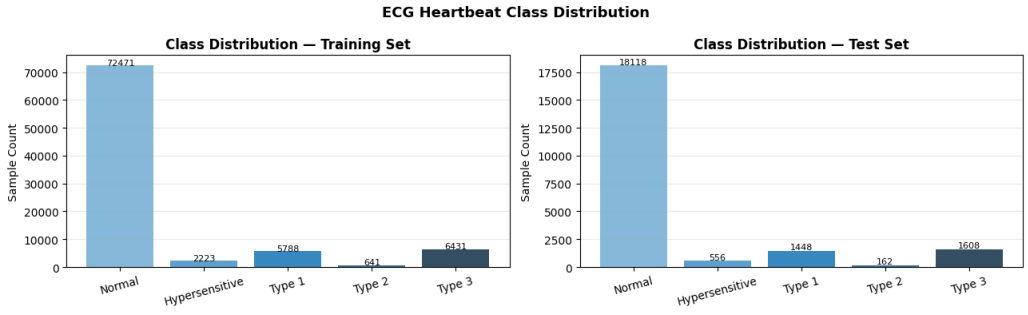


Figure 1: Class distribution in the ECG dataset.

## 2 Data Preprocessing

I started by loading the training and test CSV files and checking their shapes. The training data had shape `(87554, 188)` and the test data had shape `(21892, 188)`. After

separating the final label column, the model used 187 input features. I also checked for missing values in both files. There were no missing values, so I did not need to remove rows or fill anything in.

During exploration, I noticed that some of the later feature columns contained many zeros. I treated these as zero-padding, not corrupted data. ECG heartbeat segments can have different effective lengths, but the dataset stores every sample in a fixed-length format. Shorter signals are padded with zeros at the end. I left this alone because it is part of the fixed ECG representation, and the CNN can learn to focus on the useful part of the signal instead of the padded tail.

The class labels were converted into one-hot encoded vectors because the final layer used five Softmax outputs. This means the model predicts five probabilities, one for each class. I used categorical cross-entropy because it matches one-hot encoded multiclass classification.

I was not fully sure at first whether StandardScaler was needed because the ECG values were already mostly in a limited range. After practising this in the lab, I decided to use it anyway because it helped keep feature scaling consistent. I fitted the scaler only on the training data and then applied it to both the training and test data. This was important because fitting it on the test set would leak test information into training.

The biggest preprocessing issue was the class imbalance. At first I thought accuracy would be enough. It was not. The first failed attempt made that clear. Since most samples were Normal heartbeats, the model could score well while still missing rare classes. I used `sklearn.utils.compute_class_weight` so that rare classes had more influence during training.

Table 1: Class distribution and computed class weights.

| Class | Training Samples | Class Weight |
|---|---|---|
| Normal | 72,471 | 0.2416 |
| Hypersensitive | 2,223 | 7.8768 |
| Type 1 | 5,788 | 3.0254 |
| Type 2 | 641 | 27.3179 |
| Type 3 | 6,431 | 2.7229 |

Type 2 received the largest weight because it was the rarest class. This made mistakes on Type 2 more expensive during training. I also used shuffling before validation splitting because the dataset appeared to be ordered by class. At one point I realised that the validation split was taking the wrong portion of the data because I had not shuffled properly, and this took me a while to debug. Looking back, the shuffling step was probably the most important fix that was not obvious at first.

Finally, for the CNN, I reshaped the data from `(87554, 187)` to `(87554, 187, 1)`. Here, 187 is the ECG sequence length and 1 is the single channel. This tells the Conv1D layers to treat each heartbeat as a 1D signal rather than a flat table.

# 3 Model Development and Architecture

I did not get to the final model in one step. My first attempt was useful because it showed what was wrong with the original setup. In Attempt 1, the model reached almost 100%

training accuracy and the training loss was close to zero, but the validation accuracy was only 13.87% and the validation loss was about 42.26. At first it looked successful. It was not. The validation result showed memorisation rather than learning. That was when I realised that high training accuracy by itself does not prove the model is useful.

The reason was mainly the combination of class imbalance and a poor validation split. Without class weighting, the loss function pushed the model towards the majority Normal class. Without proper shuffling, the validation set could also contain a very different class distribution because the data appeared ordered by class. This explained why the training result looked perfect but the validation result collapsed.

In the next development attempt, I added class weights and fixed the setup. One corrected baseline reached about 86.7% training accuracy and 80.4% validation accuracy, with a gap of about 6%. This was much healthier than Attempt 1 because the model was no longer only memorising the training data.

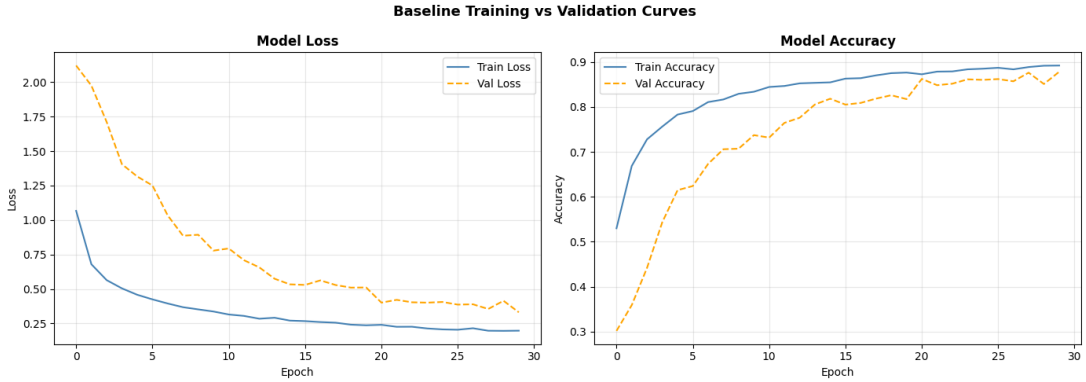


Figure 2: Corrected baseline training and validation loss/accuracy curves.

Figure 2 shows that the corrected baseline learned more sensibly than the failed first attempt. Training loss went down and validation loss followed the same general direction, although it was noisier because of the weighted loss and rare classes. The accuracy curves also moved upward together, which showed a better training pattern.

The final architecture was a 1D CNN with three convolutional blocks. The filters increased from 32 to 64 to 128 so the model could learn simple signal shapes first and more detailed patterns later. Each block used Conv1D, BatchNormalization, MaxPooling1D, and Dropout. After these blocks, I used Flatten, a Dense layer with 128 neurons, and a final Dense layer with five Softmax outputs. The final model had 376,837 parameters.

Table 2: Final 1D CNN architecture.

| Layer / Block | Purpose |
|---|---|
| Input `(187, 1)` | One ECG heartbeat signal with one channel |
| Conv1D, 32 filters | Detect simple local ECG patterns |
| BatchNorm + MaxPool + Dropout | Stabilise, reduce size, and regularise |
| Conv1D, 64 filters | Learn more detailed waveform features |
| BatchNorm + MaxPool + Dropout | Stabilise, reduce size, and regularise |
| Conv1D, 128 filters | Learn higher-level signal patterns |
| BatchNorm + MaxPool + Dropout | Stabilise, reduce size, and regularise |
| Flatten | Convert feature maps into a vector |
| Dense(128) | Combine extracted features for classification |
| Dense(5, Softmax) | Output five class probabilities |

I used Adam as the optimiser and categorical cross-entropy as the loss function. Adam is a common gradient-based optimiser, and categorical cross-entropy fits a five-class one-hot encoded problem. I used a learning rate of 0.0001 because a larger value, such as 0.001, became unstable with the strong class weights. The smaller learning rate gave the model more careful updates.

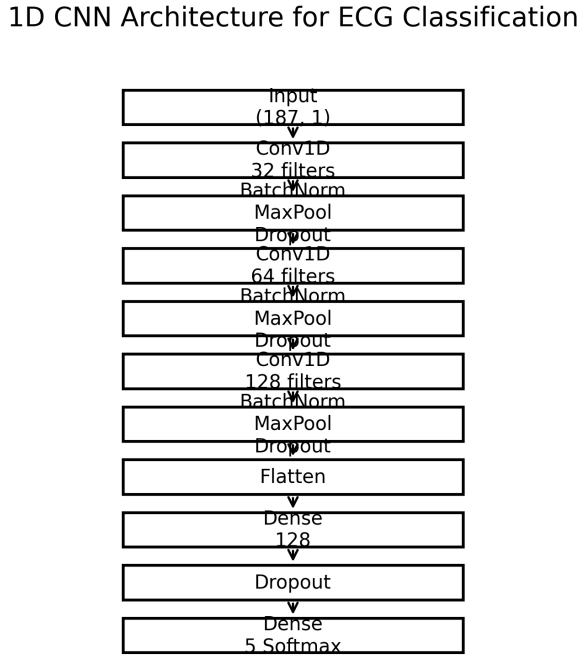


Figure 3: Final 1D CNN architecture used for ECG classification.

# 4 Training and Cross-Validation

From the workshop practice, I used EarlyStopping and ModelCheckpoint during training. EarlyStopping stopped training when validation performance stopped improving, and ModelCheckpoint saved the best model instead of only keeping the final epoch.

For tuning, I used a small manual grid search. I focused on dropout, learning rate, and batch size because these were the settings that affected this model most. I did not use the test set during tuning because that would make the final test result unfair.

Table 3: Grid search results.

| Dropout | Learning Rate | Batch Size | Validation Accuracy |
|---|---|---|---|
| 0.3 | 0.0001 | 32 | 0.8665 |
| 0.3 | 0.00005 | 32 | 0.8048 |
| 0.5 | 0.00005 | 32 | 0.5346 |
| 0.5 | 0.0001 | 32 | 0.3190 |

The best setting was dropout = 0.3, learning rate = 0.0001, and batch size = 32. Dropout of 0.5 performed worse, probably because it removed too much useful ECG information. The lower learning rate also made sense because class weights made rare-class mistakes more important, so the optimiser needed smaller updates.

After tuning, I used Stratified 5-Fold cross-validation. This means the model trained five times, each time using a different validation fold. I used StratifiedKFold instead of normal KFold because the dataset was heavily imbalanced and every fold needed a similar mix of classes.

Table 4: Stratified 5-fold cross-validation results.

| Fold | Validation Accuracy |
|---|---|
| 1 | 0.8806 |
| 2 | 0.8486 |
| 3 | 0.8915 |
| 4 | 0.8725 |
| 5 | 0.8892 |
| Mean | 0.8765 |
| Standard Deviation | 0.0155 |

The mean validation accuracy was 0.8765 with a standard deviation of 0.0155. The folds were not identical. That is normal. They were still close enough to suggest that the model was not depending on one lucky split. I also checked the loss curves because I did not want to rely only on the mean accuracy. Training loss went down in every fold, while validation loss was noisier. That made sense because the rare classes make validation less smooth.

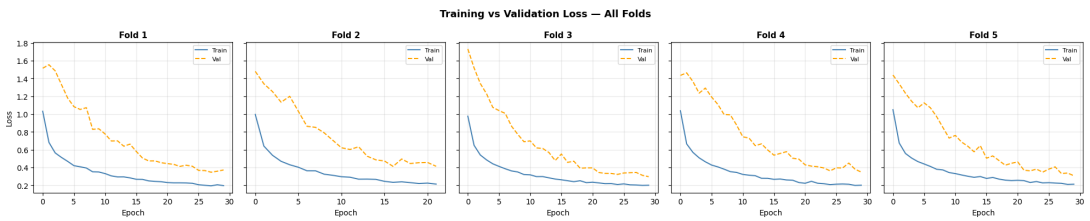


Figure 4: Training and validation loss curves across the five folds.

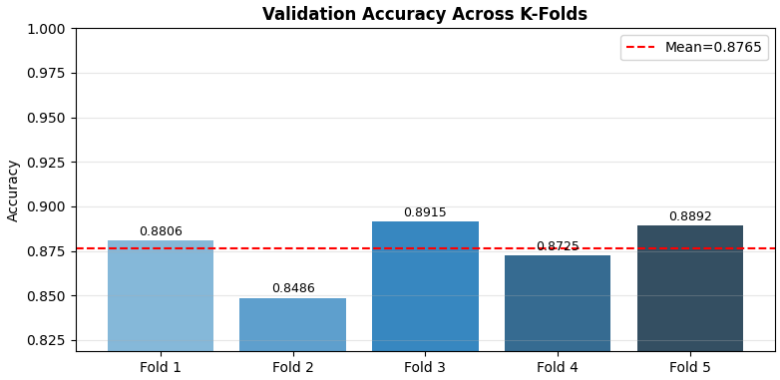


Figure 5: Validation accuracy comparison across folds.

# 5 Final Evaluation

After cross-validation, I trained the final model using the complete training set with the best hyperparameters. I preferred this over submitting one fold model because the final model could learn from all available training data after the validation setup had already been checked. I also chose to report the result that matched my final saved notebook and model file, even though an earlier run gave slightly higher accuracy. I thought this was safer because the submitted files need to be consistent.

The final model achieved 88.77% training accuracy and 87.83% test accuracy. The gap between train and test accuracy was only 0.94 percentage points. That was a good sign. It suggested that the model generalised reasonably well and did not badly overfit. The model also achieved 0.9963 train macro ROC-AUC and 0.9890 test macro ROC-AUC, so it was still separating the classes well even when accuracy was not perfect.

Table 5: Final test-set metrics.

| Class | Precision | Recall/Sensitivity | Specificity |
|---|---|---|---|
| Normal | 0.9960 | 0.8635 | 0.9833 |
| Hypersensitive | 0.2465 | 0.8849 | 0.9295 |
| Type 1 | 0.8526 | 0.9385 | 0.9885 |
| Type 2 | 0.1577 | 0.9259 | 0.9631 |
| Type 3 | 0.9623 | 0.9838 | 0.9969 |

The best part of the final model was recall. All classes had recall between about 86% and 98%. For me, this was the clearest sign that class weighting had helped. The minority classes were no longer being ignored. Type 1 and Type 3 were the strongest classes because they had both good precision and good recall.

The main weakness was precision for the rare classes. Hypersensitive had precision of 0.2465 and Type 2 had precision of 0.1577. This means the model sometimes over-predicted these rare classes. I saw this as a precision-recall tradeoff caused by class weighting. Since Type 2 had only 641 training examples, the model did not have enough real examples to learn a tight boundary. In a medical setting, this tradeoff can still make sense because missing an abnormal heartbeat is usually worse than raising a false alarm. One thing I was happy with was that specificity stayed above 0.92 even for the harder minority classes, so the model was still fairly good at recognising true negatives.

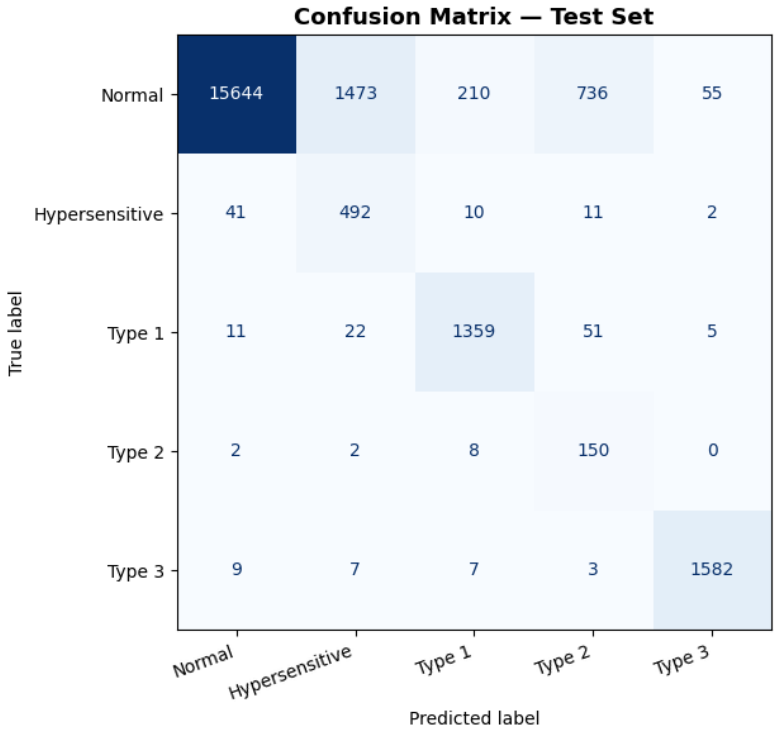


Figure 6: Confusion matrix on the test set.

Based on the confusion matrix, the model correctly classified many Normal, Type 1, and Type 3 samples. In the test set it correctly classified 15,644 Normal samples, 1,359 Type 1 samples, and 1,582 Type 3 samples. The main error pattern was Normal samples being predicted as minority abnormal classes: 1,473 Normal samples were predicted as Hypersensitive and 736 were predicted as Type 2. This explains why precision was lower for those two classes.

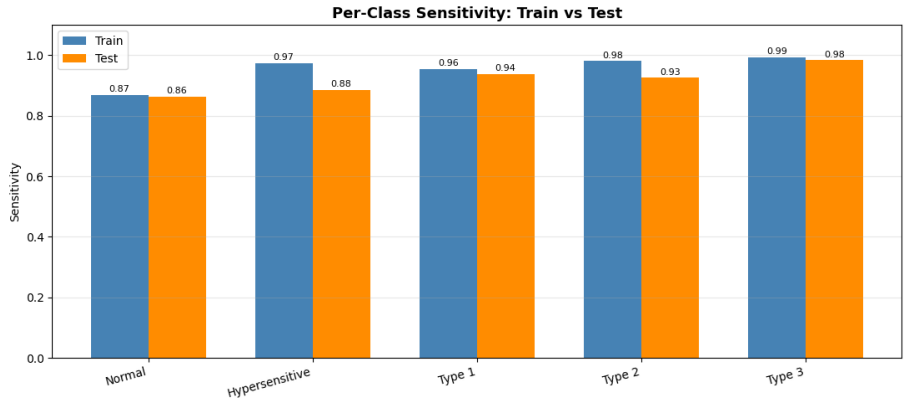


Figure 7: Train and test sensitivity comparison.

The sensitivity comparison also helped me check overfitting. Train and test sensitivities followed a similar pattern. That was useful because it suggested the model was not behaving completely differently on unseen data.

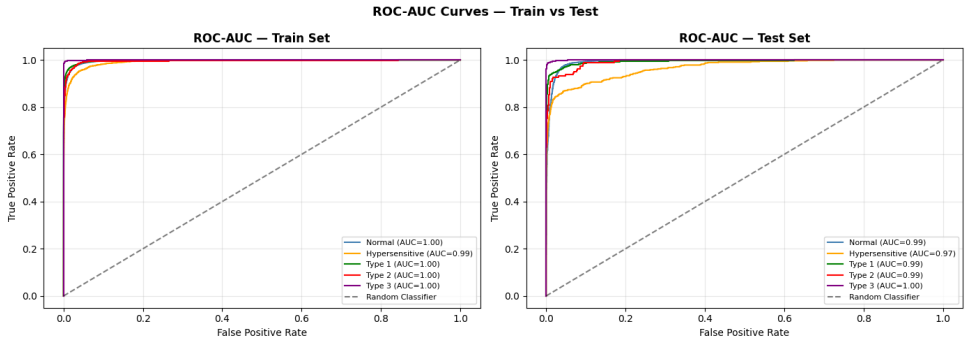


Figure 8: ROC-AUC curves for the final model.

Figure 8 shows strong ROC-AUC performance for all classes. On the test set, Normal and Type 2 reached about 0.99, Type 1 reached about 0.99, Type 3 reached 1.00, and Hypersensitive was slightly lower at 0.97. This still shows good separation, even though Hypersensitive was one of the harder classes.

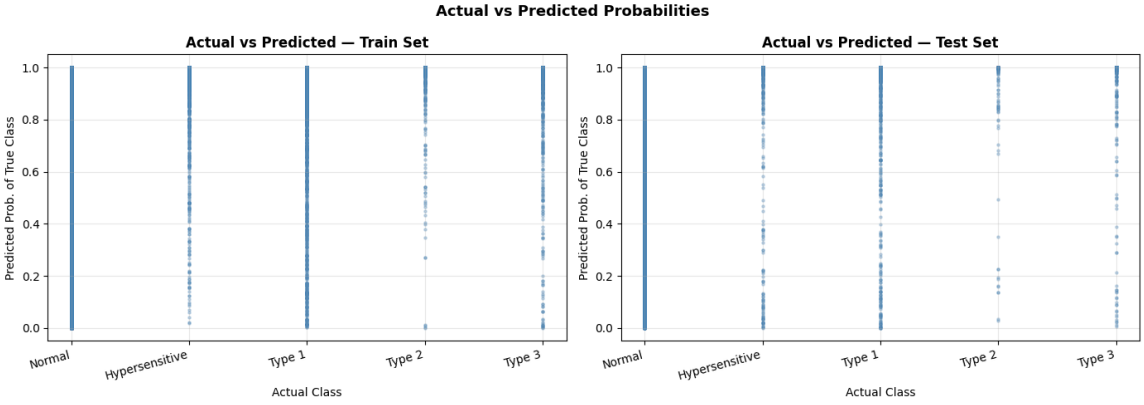


Figure 9: Actual class against predicted probability for train and test samples.

Figure 9 compares the true class with the probability assigned to that class. Most points are close to 1.0, so the model often gave high confidence to the correct class. Not always. The lower points show uncertain or wrong predictions, mainly in the minority classes. The train and test plots looked similar, which supported the idea that the model behaved consistently on unseen data.

# 6 Limitations and Development Notes

During development, I also tested SMOTE to improve minority-class precision by creating synthetic rare-class examples [5]. At first, the numbers looked better. After checking the process again, I decided it was not safe because the tuning step risked leaking test information. So I removed SMOTE from the final model.

I decided that the cleaner class-weighted CNN was safer because it used the original dataset and avoided synthetic samples. The lower precision on rare classes was therefore acknowledged instead of hidden. Type 2 had only 641 examples out of 87,554 training samples. Not enough. This is why I see the problem mainly as data scarcity, not just poor model design.

Possible improvements include collecting more minority-class ECG samples, applying L2 regularisation, tuning class-specific decision thresholds, or using a deeper CNN architecture. Because of training time, I kept the grid search small rather than testing many CNN architectures. I could have tested more combinations, but each full CNN run took time, so I kept the search small and focused on the settings that seemed to matter most. I could also compare the CNN against lecture-based models such as SVMs, Random Forests, and AdaBoost. These could be useful baselines, although they treat ECG features more like tabular data than a 1D signal. K-means could also be used for exploration, but not as a replacement because the dataset already has labels. I do not think regularisation or architecture changes alone would fully solve the rare-class precision issue because the main problem is the small number of real Type 2 samples.

# 7 Conclusion

In this coursework I built a 1D CNN to classify ECG heartbeats into five classes. The project started with a failed model that reached 100% training accuracy but only 13.87% validation accuracy. By looking into the failure, I found that the main problems were class imbalance, poor validation splitting, and unstable training choices. The final model fixed these with shuffling, StandardScaler, balanced class weights, a lower learning rate, dropout, hyperparameter tuning, and Stratified 5-Fold cross-validation.

The final matched model achieved 87.83% test accuracy, 88.77% training accuracy, a train-test gap of 0.94 percentage points, and a test macro ROC-AUC of 0.9890. These are not the highest possible numbers. But they are consistent with the submitted notebook and model file. They also show good generalisation and strong class separation. The model had high recall across all classes, which means it did not simply ignore the rare classes.

The main limitation is low precision for Hypersensitive and Type 2. This was expected because the dataset is very imbalanced, especially for Type 2. If I were to continue this work, I would first try threshold tuning for the minority classes rather than another architecture change, because I think the low precision is more about the decision boundary than the extracted features. Overall I think the approach was the right one for this dataset - not perfect, but consistent, explainable, and honest.

Progress Link: `https://github.com/sohaibkh1/ECG-CA`

# References


[1] G. B. Moody and R. G. Mark, "The impact of the MIT-BIH Arrhythmia Database," *IEEE Engineering in Medicine and Biology Magazine*, vol. 20, no. 3, pp. 45–50, 2001.

[2] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[3] S. J. D. Prince, *Understanding Deep Learning*. MIT Press, 2023.

[4] P. Rajpurkar et al., "Cardiologist-level arrhythmia detection with convolutional neural networks," arXiv preprint arXiv:1707.01836, 2017.

[5] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.